Software Requirements Specification

for

Time Travel Stock Simulator

Version 2.0 approved

Prepared by : Ayeshaani Kushwaha
Charvi Joshi
Ayush Solanki
Chirag Hoondlani
Dherya Mujawdiya

< Shri Govindram Seksaria Institute of Technology and Science>

<19/02/2026>

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 2
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 3
2.5 Design and Implementation Constraints 3
2.6 User Documentation 3
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 4
3.3 Software Interfaces 4
3.4 Communications Interfaces 4
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 5
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 6
5.4 Software Quality Attributes 6
6. Other Requirements 6
Appendix A: Glossary 7
Appendix B: Analysis Models 7
Appendix C: To Be Determined List 7

Revision History

Name	*Date*	*Reason For Changes*	*Version*

1. Introduction

1.1 Purpose

The purpose of the Time-Travel Portfolio Engine is to provide a "Gamified Historical Simulation Platform" that allows users to perform financial strategy backtesting using real-world data from the past rather than fake random numbers or delayed real-time markets.

The system is architected to teach professional-grade portfolio management—focusing on P&L, cost basis, and fees—while utilizing design patterns like the Memento Pattern to allow users to "undo" trades and analyze their decision-making processes.

1.2 Document Conventions

The following conventions are used throughout this document.

- *Bold text is used for emphasis and terminology.*
- *Arial font is used for code elements, file paths, and database fields.*
- *Headings are numbered using the format 1, 1.1, 1.2, etc.*
- *The document follows the IEEE 830-1998 standard for SRS structure.*

1.3 Intended Audience and Reading Suggestions

Intended Audience
This document is intended for:
- *Developers: To implement the backend and frontend logic.*
- *Project Managers/Faculty: To evaluate the scope and technical depth of the project.*
- *Testers: To validate the system against defined requirements.*

Reading Guide:
- *Developers should focus on Section 3 (Interfaces) and Section 4 (System Features).*
- *Evaluators should focus on Section 2 (Overall Description) and Section 5 (Non-Functional Requirements).*

1.4 Product Scope

The Time-Travel Portfolio Engine helps students understand market dynamics by allowing them to "replay" history.
- ***In Scope:***
  - *Historical Data Ingestion (CSV).*
  - *Deterministic Simulation Logic (Day-by-Day processing).*
  - *User Portfolio Management (Buy/Sell/Hold).*
  - *Algorithmic Strategy Execution (Rebalancing).*
  - *Gamification (Badges, Streaks).*
  - *Visual Analytics (Charts, Tables).*
- ***Out of Scope:***
  - *Real-time stock market integration (Live Data).*
  - *Actual monetary transactions.*
  - *Machine Learning / Predictive AI (The system is deterministic).*
  - *Mobile App development (Web only).*

1.5 References

1. *IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.*
2. *Design Patterns: Elements of Reusable Object-Oriented Software (Gamma et al.) - For Factory, Strategy, Memento, Observer patterns.*
3. *FastAPI Documentation: https://fastapi.tiangolo.com/*
4. *React.js Documentation: https://reactjs.org/*

2. Overall Description

2.1 Product Perspective

The Time-Travel Portfolio Engine is a standalone web-based application. It does not depend on external live APIs to function, ensuring reliability during offline demonstrations.

- *Client-Server Architecture: Restful API communication between React Frontend and Python Backend.*
- *Data Persistence: File-based CSV repository for market data; In-memory session storage for simulation states.*

2.2 Product Functions

The major functions of the system are:

1. *Market Data Management: Admin upload and validation of historical price data.*
2. *Portfolio Simulation: Allowing users to create fake portfolios and simulate performace over past dates.*
3. *Time Travel: The ability to "Undo" decisions or "Replay" specific date ranges.*
4. *Automated Trading: Executing pre-defined strategies (e.g., "Monthly Rebalancing").*
5. *Performance Analysis: Calculating Metrics like CAGR, Drawdown, and volatility.*
6. *Gamification: Tracking user behavior to award badges and maintain streaks.*

2.3 User Classes and Characteristics

2.3.1 The Administrator (Faculty/TA)

- *Experience: High technical proficiency.*
- *Role: Maintains the "Market". Uploads clean CSV data.*
- *Capabilities: Full access to all modules, including Data Ingestion.*

2.3.2 The Student (End User)

- *Experience: Novice to Intermediate in Finance.*
- *Role: Learner. Uses the app to test strategies.*
- *Capabilities: Restricted to "Simulation Mode". Cannot modify historical data.*

2.4 Operating Environment

- *Client: Modern Web Browser (Chrome 90+, Edge, Firefox). Resolution > 1280x720.*
- *Server: Any OS capable of running Python 3.9+ (Windows 10/11, Ubuntu 20.04+, macOS).*
- *RAM: Minimum 4GB (Recommended 8GB for large dataset processing).*

2.5 Design and Implementation Constraints

- *Strict OOP: The system must demonstrate specific design patterns (Memento, Strategy, Observer) to satisfy coursework requirements.*
- *Database: While historical market data is maintained in a structured File Repository (CSV/JSON) to ensure high performance and easy extensibility, the system shall utilize a Relational Database (e.g., SQLite or PostgreSQL) specifically for the secure storage and management of user profiles, authentication credentials, and persistent simulation metadata*
- *Deterministic: The simulation must yield identical results for identical inputs (no random seed usage).*

2.6 User Documentation

- *A "Walkthrough Tutorial" (Blind Mode) is built into the application.*
- *Tooltips are provided for all financial metrics (e.g., hovering over "Drawdown" explains the term).*

2.7 Assumptions and Dependencies

- *Assumption: Historical Data (Yahoo Finance CSVs) is clean and continuous.*
- *Dependency: The host machine has Python and Node.js installed.*
- *Dependency: Localhost ports 3000 (React) and 8000 (FastAPI) are available.*

3. External Interface Requirements

3.1 User Interfaces

The system shall provide a clean, dashboard-style interface.

- Theme: Professional Financial Dark/Light mode.
- Layout:
  - Sidebar: Navigation (Dashboard, Portfolio, Settings).
  - Top Bar: Global Stats (Net Worth, Date, User Profile).
  - Main Content: Dynamic Grid System (Charts, Tables).
- Responsiveness: Desktop-first design.

3.2 Hardware Interfaces

- *No specific hardware interfaces (sensors etc.) are required.*
- *Standard Keyboard/Mouse input.*

3.3 Software Interfaces

- *OS: Platform Independent (Python/Node).*
- *Libraries:*
  - *Pandas: For data manipulation.*
  - *FastAPI: For API Routing.*
  - *Recharts: For Data Visualization.*

3.4 Communications Interfaces

- *Protocol: HTTP/1.1 (REST API).*
- *Data Format: JSON (JavaScript Object Notation).*
- *CORS: Cross-Origin Resource Sharing enabled for local development.*

4. System Features

4.1 Authentication Module

- *FR-4.1.1: System shall present a Landing Page with "Admin" and "Student" portals.*
- *FR-4.1.2: Admin portal shall be password protected (Simple Env var check).*

4.2 Admin Data Management Module

Description: Allows management of the historical datasets.

- *Inputs: CSV File (Date, Open, Close).*
- *Processing:*
  - *Check for missing headers.*
  - *Check for non-numeric price data.*
  - *Sort by Date Ascending.*
- *Outputs: Success/Error message.*
- *FR-4.2.1: Upload CSV.*
- *FR-4.2.2: Validate Data Integrity.*
- *FR-4.2.3: List available Assets (e.g., AAPL.csv, GLD.csv).*

4.3 Simulation Engine (Core Logic)

Description: The heart of the system that advances time.

- *FR-4.3.1 (Initialization): System initializes a Portfolio object with Cash = $10,000.*
- *FR-4.3.2 (Step): On Next Day command, system retrieves price data for Current Date + 1.*
- *FR-4.3.3 (Valuation): System calculates Net Worth = Cash + (Shares * Current Price).*
- *FR-4.3.4 (Memento): System creates a Memento snapshot of the Portfolio state and pushes it to the History Stack.*
- *FR-4.3.5 (Undo): System pops the last Memento from the stack and restores the Portfolio state.*

4.4 Strategy Execution Module

Description: Automated trading logic based on the Strategy Pattern.

- *FR-4.4.1: System shall support MonthlyRebalancingStrategy.*
- *FR-4.4.2: System shall support ThresholdRebalancingStrategy (Rebalance if drift > X%).*
- *FR-4.4.3: System shall support MovingAverageStrategy (Buy if Price > 50 SMA).*
- *FR-4.4.4: User can swap strategies mid-simulation.*

4.5 Gamification & Observer Module

Description: Monitors user actions to award badges.

- *FR-4.5.1: Observer listens for MarketCrashEvent (Drop > 10%).*
- *FR-4.5.2: If User acts = "HOLD", award "Diamond Hands" Badge.*
- *FR-4.5.3: If User acts = "SELL", award "Panic Seller" Badge (Grey out Diamond badge).*
- *FR-4.5.4: Track "Winning Streak" (Consecutive days where Net Worth > Previous Net Worth).*

4.6 Dashboard Visualization Module

The dashboard consists of 10 Sub-Modules:

- *Streak Counter: Visual Fire icon + number.*
- *Badge Case: Grid of Icons.*
- *Report Card: Monthly Letter Grade (A-F).*
- *Holdings Table: Data Grid (Asset, Qty, Avg Price, LTP, P&L).*
- *Ledger: Scrolling list of Transaction Strings.*
- *Scoreboard: Big Number Cards (Net Worth).*
- *Allocation Pie: Breakdown of portfolio weights.*
- *Sector Bar: Breakdown by Industry Tag.*
- *Trading Calendar: Heatmap visualizing daily performance (Green/Red).*
- *Fees Counter: Simulated broker fee calculator.*

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- *NFR-5.1.1: The system shall load 5 years of daily data (approx 1250 records) in under 2 seconds.*
- *NFR-5.1.2: Chart re-rendering shall occur in under 60ms (60 FPS target) during interaction.*

5.2 Safety Requirements

- *NFR-5.2.1: System must handle "Divide by Zero" errors (e.g., if Asset Price = 0).*
- *NFR-5.2.2: System must prevent "Negative Cash" (User cannot buy more than they can afford).*

5.3 Security Requirements

- *NFR-5.3.1: Input sanitization on CSV uploads to prevent malicious file execution.*
- *NFR-5.3.2: No personal data storage (GDPR compliant by design - runs locally).*

5.4 Software Quality Attributes

- *Modularity: High cohesion, low coupling. Each Design Pattern creates a distinct module.*
- *Extensibility: New assets can be added effectively by dropping CSVs into the /data folder without code changes.*
- *Usability: "Blind Mode" interface must be intuitive for users with no financial background.*

6. Other Requirements

6.1 Data Schema Definitions

Asset CSV Schema:
Date (YYYY-MM-DD), Open (Float), High (Float), Low (Float), Close (Float), Volume (Int)

Memento JSON Schema:

```
{
  "id": "uuid",
  "date": "2020-01-01",
  "cash": 10000.00,
  "holdings": {
    "AAPL": 10,
    "GLD": 5
  },
  "net_worth": 12500.00
}
```

6.2 Legal & Compliance

- *Disclaimer: This software is a simulation engine only. It does not provide financial advice.*
- *License: Academic License (Open Source / MIT).*

Appendix A: Glossary

- *Asset: A tradeable financial instrument (e.g., Stock, ETF, Gold).*
- *Backtesting: Testing a trading strategy on prior time periods.*
- *Blind Mode: Running a simulation without knowing future market data.*
- *Drawdown: The measure of the decline from a historical peak in net worth.*
- *Portfolio: A collection of financial investments.*
- *Rebalancing: Realigning the weightings of a portfolio of assets.*
- *Time-Travel / Memento: The ability to undo the simulation to a previous saved state.*

Appendix B: Analysis Models

7.1 Class Diagram Description

- SimulationEngine (Singleton): Controls the clock.
- Portfolio (Originator): Holds the state.
- PortfolioMemento (Memento): Frozen state object.
- History (Caretaker): Stack of Mementos.
- IStrategy (Interface): Abstract base for trading logic.
  - CalendarStrategy (Concrete)
  - ThresholdStrategy (Concrete)
- AssetFactory (Factory): Creates Asset objects from CSV.
- AchievementObserver (Observer): Subscribes to Portfolio events.

Appendix C: To Be Determined List

- *Exact location and hosting solution for the live Demo (e.g., Vercel, Render).*
- *Final list of supported historical Assets beyond AAPL, GLD, and TSLA.*
- *Threshold parameters for grading "Report Cards" (A through F).*
- *Definition of all Achievement Badges to be implemented.*